In [1]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from torch.utils.data import Dataset, DataLoader
from pathlib import Path
from datetime import datetime
import seaborn as sns
```

Data Preparation

In [3]:
```
data = pd.read_csv('../../rnn_data.csv')
```

In [6]:
```
data.head()
```

Out[6]:

	pitch_type	game_date	batter	pitcher	description	zone	stand	p_throws	balls	s
0	FF	2025-03-18	660271	684007	called_strike	12.0	L	L	0	
1	SL	2025-03-18	660271	684007	ball	13.0	L	L	0	
2	FF	2025-03-18	660271	684007	hit_into_play	5.0	L	L	1	
3	FS	2025-03-18	669242	684007	ball	13.0	R	L	0	
4	FS	2025-03-18	669242	684007	swinging_strike	14.0	R	L	1	

5 rows × 40 columns

In [7]:
```
data["is_real_pitch"] =  data["pitch_type"].notna() & (data["pitch_type"] != "ABS")

data["target_is_real_pitch"] = data.groupby("pa_id")["is_real_pitch"].shift(-1)

data["y_next_pitch_type"] = data.groupby("pa_id")["pitch_type"].shift(-1)

data_train = data[data["target_is_real_pitch"] == True].copy()
```

In [8]:
```
# randomly select plate appearances to be a part of training and test sets

def split_by_pa_id(df: pd.DataFrame, pa_col="pa_id", ratios=(0.8, 0.2), seed: int=4
    r_train, r_test = ratios
    assert abs((r_train + r_test) - 1.0) < 1e-9

    pa_ids = df[pa_col].dropna().unique()

    rng = np.random.default_rng(seed)
    rng.shuffle(pa_ids)
```

```
    n = len(pa_ids)
    n_train = int(n*r_train)

    train_ids = set(pa_ids[:n_train])
    test_ids = set(pa_ids[n_train:])

    train_df = df[df[pa_col].isin(train_ids)].copy()
    test_df = df[df[pa_col].isin(test_ids)].copy()

    return train_df, test_df, train_ids, test_ids
```

In [9]:
```
train_df, test_df, train_ids, test_ids = split_by_pa_id(
    data_train, pa_col="pa_id", ratios=(0.8, 0.2), seed=7
)
```

In [10]:
```
# Features to be used by the model
FEATURE_SPEC = {
    "target": "y_next_pitch_type",
    "cat_cols": [
        "pitcher", "batter", "stand", "p_throws", "inning_topbot",
        "count_state", "prev_pitch_type"
    ],
    "num_cols": [
        "balls", "strikes", "outs_when_up", "inning", "score_diff_bat",
        "on_1b", "on_2b", "on_3b"
    ],
}

TARGET_COL = FEATURE_SPEC["target"]
CAT_COLS = FEATURE_SPEC["cat_cols"]
NUM_COLS = FEATURE_SPEC["num_cols"]
```

RNN Setup

In [11]:
```
PAD_ID = 0

def build_vocab(values):
    uniq = pd.Series(values.dropna().unique())
    return {v: i for i, v in enumerate(uniq, start=1)}

def encode(series, vocab):
    return series.map(vocab).fillna(PAD_ID).astype(int)

cat_vocabs = {c: build_vocab(train_df[c]) for c in CAT_COLS}
y_vocab    = build_vocab(train_df[TARGET_COL])

def encode_df(df):
    out = df.copy()
    for c in CAT_COLS:
        out[c + "_id"] = encode(out[c], cat_vocabs[c])
    out["y_id"] = encode(out[TARGET_COL], y_vocab)
    for c in NUM_COLS:
        out[c] = pd.to_numeric(out[c], errors="coerce").fillna(0).astype(np.float32
```

```
    return out

train_enc = encode_df(train_df)
test_enc  = encode_df(test_df)
```

In [12]:
```
# make all pitch sequences the same length
def make_fixed_sequences(df, pa_col="pa_id", max_len=8):
    X_cat, X_num, Y = [], [], []

    for _, g in df.groupby(pa_col, sort=False):

        cat = g[[c + "_id" for c in CAT_COLS]].to_numpy(np.int64)
        num = g[NUM_COLS].to_numpy(np.float32)
        y   = g["y_id"].to_numpy(np.int64)

        L = min(len(g), max_len)
        cat, num, y = cat[:L], num[:L], y[:L]

        pad = max_len - L
        if pad > 0:
            cat = np.pad(cat, ((0,pad),(0,0)), constant_values=PAD_ID)
            num = np.pad(num, ((0,pad),(0,0)), constant_values=0.0)
            y   = np.pad(y,   (0,pad),         constant_values=PAD_ID)

        X_cat.append(cat); X_num.append(num); Y.append(y)

    return (
        torch.tensor(np.stack(X_cat), dtype=torch.long),
        torch.tensor(np.stack(X_num), dtype=torch.float32),
        torch.tensor(np.stack(Y),     dtype=torch.long),
    )

MAX_LEN = 8
Xc_tr, Xn_tr, Y_tr = make_fixed_sequences(train_enc, max_len=MAX_LEN)
Xc_te, Xn_te, Y_te = make_fixed_sequences(test_enc,  max_len=MAX_LEN)
```

In [13]:
```
# Create the Dataset
class PitchSeqDS(Dataset):
    def __init__(self, Xc, Xn, Y):
        self.Xc, self.Xn, self.Y = Xc, Xn, Y
    def __len__(self): return self.Y.size(0)
    def __getitem__(self, i): return self.Xc[i], self.Xn[i], self.Y[i]

train_loader = DataLoader(PitchSeqDS(Xc_tr, Xn_tr, Y_tr), batch_size=64, shuffle=Tr
test_loader  = DataLoader(PitchSeqDS(Xc_te, Xn_te, Y_te), batch_size=64, shuffle=Fa
```

In [14]:
```
# Model Creation
class SimplePitchRNN(nn.Module):
    def __init__(self, cat_vocab_sizes, num_features, emb_dim=16, hidden=128, num_c
        super().__init__()
        self.cat_cols = list(cat_vocab_sizes.keys())

        self.embs = nn.ModuleDict({
            col: nn.Embedding(cat_vocab_sizes[col], emb_dim, padding_idx=pad_id)
            for col in self.cat_cols
```

```
        })

        in_dim = len(self.cat_cols) * emb_dim + num_features
        self.rnn = nn.RNN(in_dim, hidden, batch_first=True)
        self.fc  = nn.Linear(hidden, num_classes)

    def forward(self, x_cat, x_num):
        embs = []
        for j, col in enumerate(self.cat_cols):
            embs.append(self.embs[col](x_cat[:, :, j]))
        x = torch.cat(embs + [x_num], dim=-1)
        h, _ = self.rnn(x)
        return self.fc(h)

cat_vocab_sizes = {c: len(cat_vocabs[c]) + 1 for c in CAT_COLS}
num_classes = len(y_vocab) + 1

# Model Initialization
model = SimplePitchRNN(cat_vocab_sizes, num_features=len(NUM_COLS), num_classes=num
```

In [15]:

```
from datetime import datetime
from pathlib import Path
import pandas as pd

# Export vocabularies and feature lists with a date-stamped filename.
today = datetime.now().strftime("%Y%m%d")
repo_root = Path.cwd().parent
vocab_dir = repo_root / "model_shared" / "vocab"
feature_dir = repo_root / "model_shared" / "feature-list"
vocab_dir.mkdir(parents=True, exist_ok=True)
feature_dir.mkdir(parents=True, exist_ok=True)

rows = []
for feature, vocab in cat_vocabs.items():
    for value, idx in vocab.items():
        rows.append({"feature": feature, "value": value, "id": idx, "kind": "catego
for value, idx in y_vocab.items():
    rows.append({"feature": TARGET_COL, "value": value, "id": idx, "kind": "target"

vocab_path = vocab_dir / f"rnn_vocab_{today}.csv"
pd.DataFrame(rows).to_csv(vocab_path, index=False)

feature_rows = []
for c in CAT_COLS:
    feature_rows.append({"feature": c, "kind": "categorical"})
for c in NUM_COLS:
    feature_rows.append({"feature": c, "kind": "numerical"})
feature_rows.append({"feature": TARGET_COL, "kind": "target"})

feature_path = feature_dir / f"rnn_vocab_{today}.csv"
pd.DataFrame(feature_rows).to_csv(feature_path, index=False)

print(f"Wrote vocab to {vocab_path}")
print(f"Wrote feature list to {feature_path}")
```

```
Wrote vocab to c:\Users\sethb\Baseball\Senior Project\Model Reports\model_shared\voc
ab\rnn_vocab_20260209.csv
Wrote feature list to c:\Users\sethb\Baseball\Senior Project\Model Reports\model_sha
red\feature-list\rnn_vocab_20260209.csv
```

Training and Evaluation

In [16]:

```python
# Simplified the training loop
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
criterion = nn.CrossEntropyLoss(ignore_index=PAD_ID)
optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 20
for epoch in range(epochs):
    model.train()
    epoch_loss = 0
    for x_cat, x_num, y in train_loader:
        x_cat = x_cat.to(device)
        x_num = x_num.to(device)
        y     = y.to(device)
        logits = model(x_cat, x_num)

        loss = criterion(
            logits.reshape(-1, num_classes),
            y.reshape(-1)
        )

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()

    print(f'Epoch [{epoch+1}/{epochs}], Loss: {epoch_loss / len(train_loader):.4f}'
```

```
Epoch [1/20], Loss: 1.6891
Epoch [2/20], Loss: 1.4550
Epoch [3/20], Loss: 1.3667
Epoch [4/20], Loss: 1.3250
Epoch [5/20], Loss: 1.3009
Epoch [6/20], Loss: 1.2844
Epoch [7/20], Loss: 1.2729
Epoch [8/20], Loss: 1.2638
Epoch [9/20], Loss: 1.2563
Epoch [10/20], Loss: 1.2498
Epoch [11/20], Loss: 1.2447
Epoch [12/20], Loss: 1.2390
Epoch [13/20], Loss: 1.2349
Epoch [14/20], Loss: 1.2311
Epoch [15/20], Loss: 1.2274
Epoch [16/20], Loss: 1.2237
Epoch [17/20], Loss: 1.2205
Epoch [18/20], Loss: 1.2172
Epoch [19/20], Loss: 1.2145
Epoch [20/20], Loss: 1.2115
```

In [17]:
```python
out_dir = Path.cwd() / "trained-parameters"
out_dir.mkdir(parents=True, exist_ok=True)
filename = f"simple_rnn_v0_1_{datetime.now().strftime('%Y%m%d')}.pt"
save_path = out_dir / filename
torch.save(model.state_dict(), save_path)
print(f"Wrote model parameters to {save_path}")
```

```
Wrote model parameters to c:\Users\sethb\Baseball\Senior Project\Model Reports\V1\trained-parameters\simple_rnn_v0_1_20260209.pt
```

In [18]:
```python
PAD_ID = 0

model.eval()
correct = 0
total = 0

with torch.no_grad():
    for x_cat, x_num, y in test_loader:
        x_cat = x_cat.to(device)
        x_num = x_num.to(device)
        y     = y.to(device)

        outputs = model(x_cat, x_num)
        predicted = outputs.argmax(dim=-1)

        mask = (y != PAD_ID)
        correct += ((predicted == y) & mask).sum().item()
        total += mask.sum().item()

accuracy = 100 * correct / total
print(f"Token Accuracy (no PAD): {accuracy:.2f}%")
```

```
Token Accuracy (no PAD): 43.78%
```

In [19]:
```python
from collections import Counter
```

```
model.eval()
counts = Counter()

with torch.no_grad():
    for x_cat, x_num, y in test_loader:
        x_cat = x_cat.to(device)
        x_num = x_num.to(device)

        preds = model(x_cat, x_num).argmax(dim=-1)
        for p in preds.view(-1).tolist():
            if p != 0:
                counts[p] += 1

print(counts.most_common(5))
```

```
[(2, 135029), (4, 34509), (1, 25028), (9, 20578), (6, 15385)]
```

In [20]:
```
# reverse vocab
id_to_pitch = {v: k for k, v in y_vocab.items()}

for pid, cnt in counts.most_common(5):
    print(id_to_pitch.get(pid, "PAD"), cnt)
```

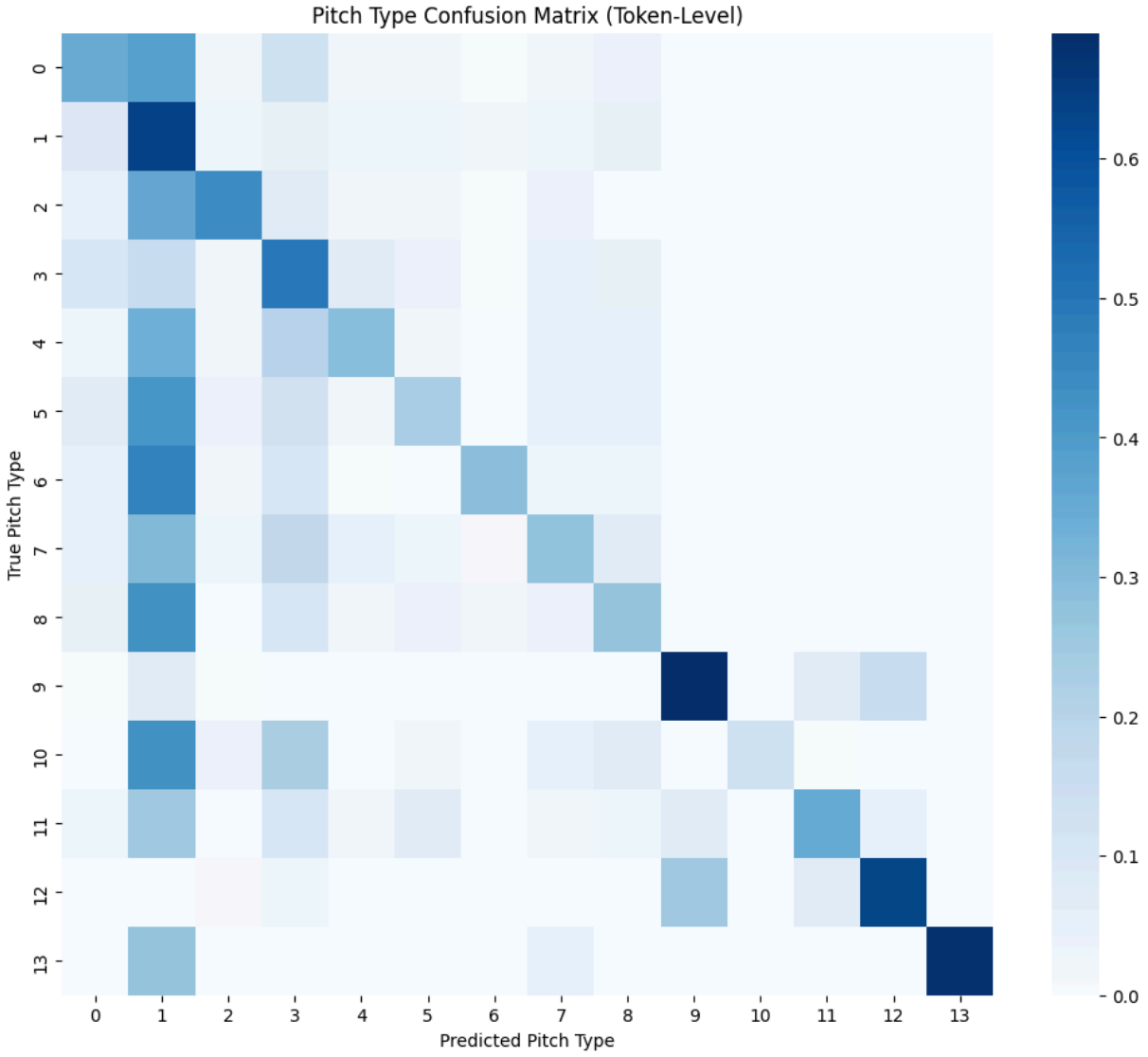
```
FF 135029
SI 34509
SL 25028
CH 20578
CU 15385
```

In [21]:
```
model.eval()
correct = 0
total = 0
K = 3

with torch.no_grad():
    for x_cat, x_num, y in test_loader:
        x_cat, x_num, y = x_cat.to(device), x_num.to(device), y.to(device)
        logits = model(x_cat, x_num)
        topk = logits.topk(K, dim=-1).indices

        mask = (y != PAD_ID)
        match = (topk == y.unsqueeze(-1)).any(dim=-1)

        correct += (match & mask).sum().item()
        total += mask.sum().item()

print(f"Top-{K} Accuracy: {100*correct/total:.2f}%")
```

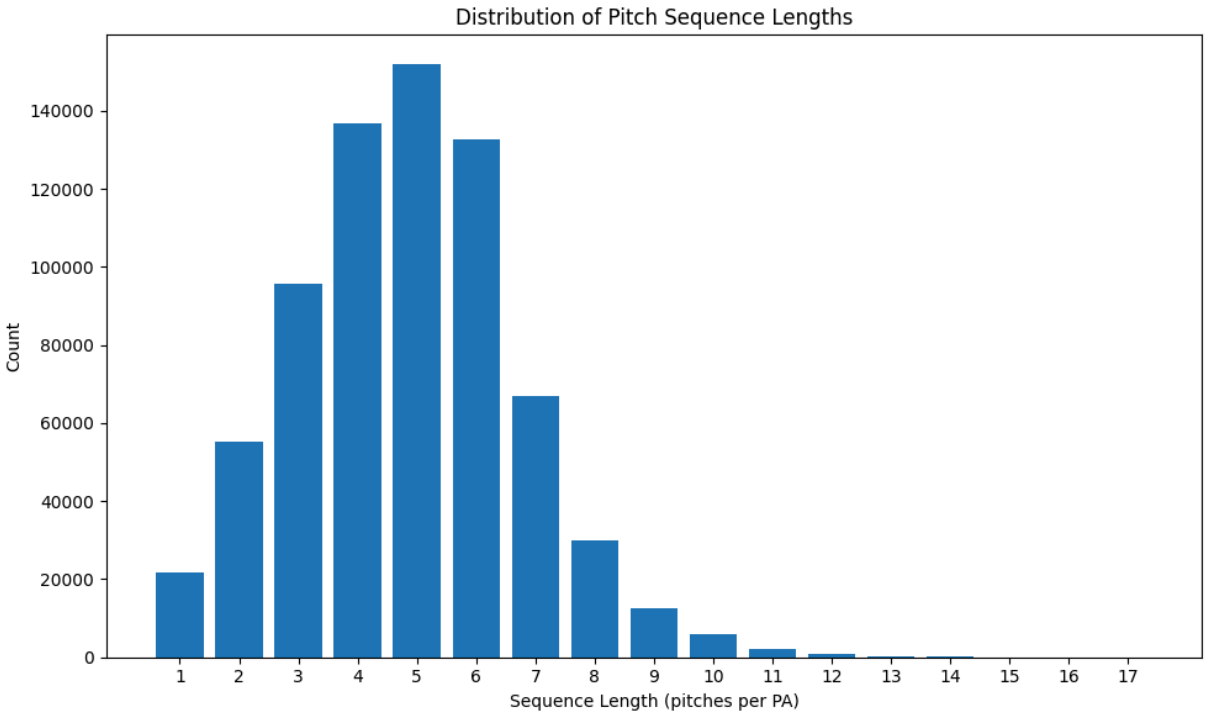
```
Top-3 Accuracy: 87.66%
```

In [22]:
```
# Confusion Matrix
from sklearn.metrics import confusion_matrix

model.eval()
all_preds = []
all_true = []
```

```
with torch.no_grad():
    for x_cat, x_num, y in test_loader:
        x_cat = x_cat.to(device)
        x_num = x_num.to(device)
        y = y.to(device)
        logits = model(x_cat, x_num)        # (B, L, C)
        preds  = logits.argmax(dim=-1)      # (B, L)

        mask = (y != PAD_ID)

        # mask == 1 for valid tokens, 0 for PAD
        valid = mask.bool()

        all_preds.append(preds[valid].cpu().numpy())
        all_true.append(y[valid].cpu().numpy())

y_pred = np.concatenate(all_preds)
y_true = np.concatenate(all_true)


cm = confusion_matrix(y_true, y_pred)
cm_norm = cm / cm.sum(axis=1, keepdims=True)

plt.figure(figsize=(12, 10))
sns.heatmap(
    cm_norm,
    cmap="Blues",
    fmt="d"
)
plt.xlabel("Predicted Pitch Type")
plt.ylabel("True Pitch Type")
plt.title("Pitch Type Confusion Matrix (Token-Level)")
plt.show()
```



In [ ]:
```
seq_counts = data["seq_len"].value_counts()

plt.figure(figsize=(10, 6))

plt.bar(seq_counts.index, seq_counts.values)

plt.xlabel("Sequence Length (pitches per PA)")
plt.ylabel("Count")
plt.title("Distribution of Pitch Sequence Lengths")

plt.xticks(seq_counts.index)  # ensures each integer shows
plt.tight_layout()
plt.show()
```



In [29]:
```
from sklearn.metrics import classification_report
report = classification_report(y_true, y_pred, target_names=list(id_to_pitch.values
print(report)
```

```
              precision    recall  f1-score   support

          SL       0.44      0.35      0.39     15820
          FF       0.47      0.64      0.54     33475
          FS       0.40      0.44      0.42      3889
          SI       0.43      0.50      0.46     15342
          ST       0.41      0.30      0.34      7596
          CU       0.36      0.23      0.28      7114
          KC       0.37      0.29      0.32      1781
          FC       0.36      0.27      0.31      7658
          CH       0.40      0.27      0.32     12150
          FA       0.74      0.69      0.72       142
          SV       0.31      0.14      0.19       530
       OTHER       0.40      0.35      0.37        71
          EP       0.71      0.63      0.67       103
          FO       0.44      0.68      0.53       112

    accuracy                           0.44    105783
   macro avg       0.45      0.41      0.42    105783
weighted avg       0.43      0.44      0.42    105783
```

In [27]:
```
from sklearn.metrics import accuracy_score
def strategic_accuracy(y_true, y_pred, id_to_pitch):
    fastballs = {'FF', 'SI', 'FC'}
    breaking = {'SL', 'CU', 'KC', 'SV', 'ST'}
    offspeed = {'CH', 'FS'}

    def pitch_to_family(pitch):
        if pitch in fastballs: return 'fastball'
```

```
        if pitch in breaking: return 'breaking'
        if pitch in offspeed: return 'offspeed'
        return 'other'

    y_true_family = [pitch_to_family(id_to_pitch[y]) for y in y_true]
    y_pred_family = [pitch_to_family(id_to_pitch[p]) for p in y_pred]

    report = classification_report(y_true_family, y_pred_family)
    print(report)

strategic_accuracy(y_true, y_pred, id_to_pitch)
# print(f"Strategic Accuracy: {strategic_acc:.2%}")
```

```
              precision    recall  f1-score   support

    breaking       0.49      0.36      0.41     32841
    fastball       0.60      0.73      0.65     56475
    offspeed       0.40      0.31      0.35     16039
       other       0.73      0.79      0.76       428

    accuracy                           0.55    105783
   macro avg       0.56      0.55      0.55    105783
weighted avg       0.53      0.55      0.53    105783
```

Model Report

Features

What features did we use?

target: "y_next_pitch_type"

cat_cols: "pitcher", "batter", "stand", "p_throws", "inning_topbot","count_state", "prev_pitch_type"

num_cols: "balls", "strikes", "outs_when_up", "inning", "score_diff_bat", "on_1b", "on_2b", "on_3b"

All these features were selected to create a very generic representation of a specific game state. count_state nad prev_pitch_type are the only two features not directly taken from the Statcast dataset.

- count_state is a string representation of the current count. In my learnings of how to implement models, one suggestion I got was to try and condense features when appropriate to avoid unnecssary extra features. I'd like to apply this same process to the base states in the future.
- prev_pitch_type simply is what the previous pitch was in an at bat. There were some extra calculations that needed to be done for this though, such as handling the start of the AB and what to do about ABS calls. Since ABS calls progress the count but don't

involve the pitcher throwing, I created a temporary feature to track the last real pitch and use that as the previous pitch whenever there is a pitch 1 -> ABS -> pitch 2.

The last feature of note is the target variable, y_next_pitch_type. The y is meant to indicate that this is the variable to train on

Some notable exclusions:

- Pitch Location / Zone - This will be added as a feature in the future, as we want to be recommeding locations
- Pitch Velocity, Spin Rate, Pitch Break - These are features which we cannot realistically predict
- Historical Outcomes - We plan to take these features into consideration in future models as well

Model Architecture Decisions

What design decisions were made?

- RNN Type
  - Started off with a very bare bones RNN to give ourselves a baseline before tweaking the design of the RNN for future models.
- RNN Layers
  - Kept things to just the embedding layer, a standard RNN layer which processes the sequence, and a Linear Layer which projects the RNN hidden state down to the 14-dimension logits (one per pitch class)
- Embedding Dimension
  - All features were given an embed_dim = 16. In the future, I'd like to play around with different embedding sizes for various features to see what the impact might be (if any). Specifically, does having a higher embed_dim for larger features such as batter or pitcher help improve the model's learning capabilites?
- Sequence Length
  - The length of each sequence was truncated to be 8 pitches. The mean seq_len was about 4.8 pitches, and based on the chart above the bast majority of ABS are 8 or less, so thats why this value was chosed.
- Padding
  - PyTorch requires all inputs be the same length prior to training. On that note, another approach to padding could be to set the max sequence length to be 16, then pad based on that to include every AB.
- Train/Test Split Strategy
  - For this model, I chose to group each plate appearance and give them a pa_id (plate appearance id). Then from this list of IDs, I randomly select 80% to be in the training set, and 20% to be in the test set. This seemed like a standard way to do things at first, but I wonder what the trade offs would be to split based on time

instead. For example, ABs in April-July are used for training, and August-September ABs are used for testing.

- Loss Function
  - CrossEntropyLoss is a standard loss funciton for multi class classification, and currently does not apply any class weights (will discuss more about this later)

Training Hyperparameters:

- Epochs: 20
- Batch Size: 64
- Train/Test Split: 80/20

Model Output

How'd the model do?

Accuracy: 43.78% Top-3 Accuracy: 87.66%

These values give a very general overview of the models performance. The model correctly picked the next pitch type 43.78% of the time, which given the context of the model isn't too bad. There are 13 pitch types the model can consider, so if the model was randomly selecting the right pitch we'd expect the accuracy to be only be about 7%. The Top-3 accuracy shows that the true pitch occured in the top 3 predictions 87% of the time, which initially sounds nice but the large gap between two values makes it seem like the model is uncertain about what the true pitch should be.

Now lets look at how individual pitches performed. Starting off with a confusion matrix, we can immediately see an overprediction of fastballs. This is not big suprise, as fastballs are the most common pitch in baseball. Our model currently does not have any class weights in place to deter the model from choosing fastballs as the next predicted pitch. 4 seam fastballs have a precision of 0.47 and recall of 0.64, which is solid considering the total number of fastballs.

This pitch by pitch report highlights a major issue with the model in its current state, which is the inability to predict breaking balls and off speed pitches. Several of those pitch types have very low recalls, which mean the model is struggling to pick up on when to throw a breaking ball. To investigate this issue further, I categorized the pitch types into fastballs, breaking, and offspeed pitches to get a better sense of what is happening overall.

As we could see from the pitch by pitch, the "pitch family" report highlights just how off the model is on predicting breaking and offspeed pitches, missing about 65% of the pitches each.

Future Improvements

1. Add dropout layers to the RNN Definition
2. Implement Early Stopping
3. Consider adding normalization to the Numerical Features
4. Switch to a class weighted loss function
5. Add different embedding dimensions for different categorical features.

For Model V2, I am specifically looking to see an improvement in predicting breaking balls and offspeed pitches. These are crucial to a pitches arsenal to compliment their fastball, so helping our model be more confident in those recommendations is important.

Long Term Improvements will inclide

- using other types of RNNs like a LSTM or GRU
- adding more layers, but not until we have a better understanding of how adding layers will impact the model's performance.
- Add more batter specific features
- Tune the Hyperparameters
- Consider adding some new features for historical data
    - pitcher usage rates
    - count state feature improvements (pitcher ahead or batter ahead)
    - Sequenctional context features (previous pitch was fastball, breaking or offspeed)